

CSCE 636 DEEP LEARNING · MODULE 06 · LECTURE NOTES

Sequences and Attention

Why recurrence lost.

MODULE OUTCOMES

Explain recurrent networks and their two fatal problems.

Explain LSTM gating and what it fixes.

Derive attention and explain each term.

Explain why attention replaced recurrence.

State attention's cost and its consequences.

1 Recurrence

AN RNN APPLIES THE SAME FUNCTION AT EVERY STEP

$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ — a hidden state carried forward through the sequence, updated by the same learned function at every position.

So the parameter count is independent of sequence length, and the same dynamics are applied everywhere. **That is exactly the right inductive bias for a sequence** (Module 01 §2), and it is why recurrence was the obvious and correct-seeming approach for decades.

Training is backpropagation through time: unroll the recurrence into a feedforward graph with one copy of f per timestep, and apply Module 02's machinery to the unrolled graph.

And the unrolled graph is as deep as the sequence is long. Both of recurrence's fatal problems come from that single fact — which is a good illustration of how an architectural choice determines a set of consequences that look unrelated.

Problem	Cause	Consequence
Vanishing (and exploding) gradients	The same Jacobian multiplied T times — Module 02 §3's product, with T in the hundreds or thousands.	Long-range dependence cannot be learned. A word at position 5 cannot influence a prediction at position 500 because no gradient survives the journey.
No parallelism	Step t requires step $t-1$, by definition. The computation is inherently sequential.	Training time scales with sequence length , and a GPU with thousands of cores processes one element at a time. This is the one that actually ended recurrence.

LSTM GATING IS THE RESIDUAL CONNECTION, IN TIME

An LSTM carries a cell state that is updated *additively*, with learned gates controlling what is written into it, what is forgotten from it, and what is read out.

The additive path means the gradient can flow backward across many timesteps with a multiplicative factor near one — which is precisely the residual connection mechanism of Module 02 §3, discovered years earlier in a different setting and for a different stated reason. **The same structural idea, arrived at twice.**

So long-range dependence became learnable, and LSTMs and GRUs dominated sequence modelling for roughly a decade — machine translation, speech recognition, language modelling.

But the sequential dependency remains, and gating does nothing about it. A GPU with thousands of cores sits almost entirely idle while a sequence is processed one element at a time (CSCE 735 Module 08) — the hardware got enormously wider and the architecture could not use it. **That is what attention removed, and it is why the replacement was so complete.**

2 Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k}) \mathbf{V}$$

Every term has a reason. $\mathbf{Q}\mathbf{K}^T$ computes a relevance score between every query and every key. The softmax turns those scores into weights that sum to one. Multiplying by $\mathbf{V}$ produces a weighted average of the values. **And the $\sqrt{d_k}$ divisor is not cosmetic:** dot products of random vectors grow in magnitude with the dimension, so without the scaling the softmax saturates at large d , its gradients vanish (Module 02 §3), and training fails. **Omitting it is a real bug that only manifests at large model dimension**, which makes it hard to catch in a small test.

A SOFT DICTIONARY LOOKUP

```
QUERY "what am I looking for?"      from the current position
KEY   "what do I offer?"            from every position
VALUE "what do I contribute if chosen?"
```

A hard lookup takes the single best-matching key.
Attention takes a WEIGHTED AVERAGE of all values, weighted by query-key similarity. Soft, and therefore differentiable.

SELF-ATTENTION: Q, K and V all come from the same sequence, so every position attends to every other, including itself.

WHY IT BEAT RECURRENCE

- * PATH LENGTH between any two positions is 1, not T.
No multiplied Jacobians, so no vanishing over distance.
- * FULLY PARALLEL -- all positions at once, as one large matrix multiply, which is exactly what GPUs do best (CSCE 735 M09).
- * relationships are LEARNED, not fixed by the architecture.

THE COST: $O(T^2)$ in time and memory.

3 The quadratic cost

QUADRATIC COST IS THE DEFINING CONSTRAINT

Every pair of positions is scored, so the cost grows as T^2 in both time and memory. Doubling the context length quadruples both.

That is why context windows were historically small and why extending them is a research area rather than a configuration change — the constraint is structural, not an implementation limit.

FlashAttention does not change the asymptotics. It avoids ever materialising the full $T \times T$ attention matrix in memory, by tiling the computation and recomputing what it needs — **which is a memory-bandwidth optimisation** (CSCE 735 Module 09 §1, exactly the tiling argument) **and a very large practical win**, enabling far longer contexts at the same hardware. It is still quadratic in arithmetic.

And the approximate variants — **sparse attention, linear attention, sliding windows** — **trade exactness for scale**, with genuinely mixed results: many report strong benchmark numbers and underperform exact attention on careful comparison. **Exact attention with better memory behaviour has mostly won**, which is a recurring pattern (CSCE 735 Module 09: fix the memory traffic before changing the algorithm).

4 What attention assumes

ATTENTION IS A WEAKER PRIOR, DELIBERATELY

Convolution asserts that nearby things relate and that position does not change meaning. Recurrence asserts that order matters and that the dynamics are uniform across the sequence. Both are strong, specific claims about the data.

Attention asserts only that *some* pairs of positions relate, and learns which ones. It imposes almost no structure — so little that **positional information has to be added back explicitly**, because the operation is otherwise entirely permutation-equivariant (Module 07 §2).

So it needs more data and scales further, which is Module 01 §2's trade at its sharpest: the weakest useful prior, maximum data appetite, and the highest ceiling.

And that is why the same architecture now works on text, images, audio, video, point clouds, and protein sequences. A prior weak enough to be wrong about nothing is also general enough to apply to everything — which is the single sentence that explains the transformer's spread across every domain in the space of a few years, and is the most important idea in this module.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Karpathy — 'Let's build GPT' (free)	Attention built from scratch on video , with every design choice explained as it is typed. The best resource for §2.
Bahdanau, Cho & Bengio — Neural Machine Translation by Jointly Learning to Align and Translate (free)	Attention's first appearance , bolted onto an RNN. Worth reading to see the problem it was invented to solve.
Christopher Olah — Understanding LSTM Networks (free)	The clearest explanation of §1's gating that exists, with diagrams that actually help.
Dao et al. — FlashAttention (free)	The §3 memory argument — and a direct application of CSCE 735 Module 09's tiling to a problem in this course.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a vanilla RNN and train it on a long-range dependency task.
- 2 **Measure how far back it can learn** by varying the dependency distance.
- 3 Implement an LSTM and repeat. Plot the achievable distance for each.
- 4 **Measure training throughput against sequence length** for an RNN and for attention on the same hardware.
- 5 Implement scaled dot-product attention from scratch.
- 6 **Remove the $\sqrt{d_k}$ scaling** and show training fails at large model dimension while succeeding at small.
- 7 **Visualise the attention weights** on a real sequence and look for interpretable patterns.
- 8 Measure attention's time and memory against sequence length and confirm the quadratic growth.
- 9 **Implement a simple tiled attention** that avoids materialising the full matrix, and measure the memory saving.
- 10 Compare a sliding-window approximation against exact attention on a task needing long-range dependence.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the RNN recurrence and say where both its problems come from.
- 2 What does LSTM gating fix, and what mechanism does it share with residual connections?
- 3 What does gating not fix, and why was that decisive?

- 4 Write the attention equation and explain every term.
- 5 What are queries, keys, and values?
- 6 Give three reasons attention beat recurrence.
- 7 Why is attention quadratic, and what does FlashAttention actually change?
- 8 Why is attention a weaker prior, and what follows from that?